

This handout includes space for every question that requires a written response. Please feel free to use it to handwrite your solutions (legibly, please). If you choose to typeset your solutions, the `README.md` for this assignment includes instructions to regenerate this handout with your typeset $\text{\LaTeX}$ solutions.

1.a

Given the following diagram:

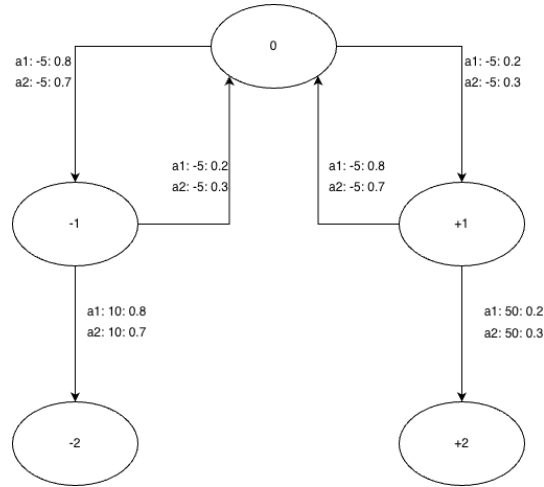


Figure 1: State Diagrams with probability (T) and Rewards (R), with $a1 = T:R$ and $a2 = T:R$

From the slides i the value is computed as follows:

$$V_i(s) = \max_{a \in A(s)} Q_i(s, a), \quad Q_i(s, a) = \sum_{s'} T(s, a, s') (R(s, a, s') + V_{i-1}(s')).$$

And the following assumptions are taken into account for the first iteration:

$$V_0^*(s) = 0 \quad \forall s \in \{-2, -1, 0, +1, +2\}, \quad V_i^*(s_{\text{end}}) = 0 \quad \forall i$$

Let's compute the first Values for Iteration 1:

$$V_1^*(-2) = 0$$

$$V_1^*(-1) = \max\{0.8 \cdot 10 + 0.2 \cdot (-5), 0.7 \cdot 10 + 0.3 \cdot (-5)\} = 7$$

$$V_1^*(0) = \max\{-5, -5\} = -5$$

$$V_1^*(+1) = \max\{0.8 \cdot (-5) + 0.2 \cdot 50, 0.7 \cdot (-5) + 0.3 \cdot 50\} = 11.5$$

$$V_1^*(+2) = 0$$

$$V_1^* = \{-2 : 0, -1 : 7, 0 : -5, +1 : 11.5, +2 : 0\}$$

After the first values of Iteration 1 computed, now let's compute Iteration 2:

$$V_2^*(-2) = 0$$

$$V_2^*(-1) = \max\{0.8 \cdot (10 + 0) + 0.2 \cdot (-5 + -5), 0.7 \cdot (10 + 0) + 0.3 \cdot (-5 + -5)\} = 6$$

$$V_2^*(0) = \max\{0.8 \cdot (-5 + 7) + 0.2 \cdot (-5 + 11.5), 0.7 \cdot (-5 + 7) + 0.3 \cdot (-5 + 11.5)\} = 3.35$$

$$V_2^*(+1) = \max\{0.8 \cdot (-5 + -5) + 0.2 \cdot (50 + 0), 0.7 \cdot (-5 + -5) + 0.3 \cdot (50 + 0)\} = 8$$

$$V_2^*(+2) = 0$$

$$V_2^* = \{-2 : 0, -1 : 6, 0 : 3.35, +1 : 8, +2 : 0\}$$

As can be seen, in Iteration 2 the values obtained in Iteration 1 are used, while in Iteration 1 all the values come from V_0 , where every state has value 0.

1.b

Intuitively, the optimal policy chooses the action in each state that gives the highest expected return based on V_2^* , meaning a_1 is preferred in -1 because it is more likely to reach the good terminal state -2 , while a_2 is preferred in 0 and $+1$ because it increases the chance of reaching the high reward terminal state $+2$.

2.a

Original MDP: For $0 < \lambda < 1$, the optimal value satisfies

$$V^*(s) = \max_{a \in A(s)} \sum_{s' \in S} T(s, a, s') \left(R(s, a, s') + \lambda V^*(s') \right).$$

New MDP: Create an absorbing state o and set $S' = S \cup \{o\}$ with discount $\lambda' = 1$ and $V'^*(o) = 0$. Then

$$V'^*(s) = \max_{a \in A(s)} \left[\sum_{s' \in S} T'(s, a, s') \left(R'(s, a, s') + V'^*(s') \right) + T'(s, a, o) \left(R'(s, a, o) + V'^*(o) \right) \right].$$

With probability λ we continue in S (like the original MDP), and with probability $1 - \lambda$ we terminate in o . Meaning of the new transition and reward terms.

- $T'(s, a, s')$: probability of moving from state s to another original state s' after taking action a . This is scaled by λ because the process only continues within the original state space with probability λ .
- $T'(s, a, o)$: probability of transitioning from state s to the absorbing state o after taking action a . This equals $1 - \lambda$ and represents the chance that the episode terminates, which replaces discounting.
- $R'(s, a, s')$: reward obtained when transitioning from s to another original state s' . This reward is scaled by $1/\lambda$ to compensate for the λ factor moved into the transition probabilities.
- $R'(s, a, o)$: reward obtained when transitioning from s to the absorbing state o . This is set to 0 because no further rewards are collected after termination.

Final definitions: For all $s \in S$, $a \in A(s)$, and $s' \in S$:

$$\begin{aligned} T'(s, a, s') &= \lambda T(s, a, s'), & T'(s, a, o) &= 1 - \lambda, \\ R'(s, a, s') &= \frac{1}{\lambda} R(s, a, s'), & R'(s, a, o) &= 0, & V'^*(o) &= 0. \end{aligned}$$

So if we inject the new terms into the original equation to recover the original and proof the values, we get the following:

$$V'^*(s) = \max_{a \in A(s)} \left[\sum_{s' \in S} T'(s, a, s') \left(R'(s, a, s') + V'^*(s') \right) + T'(s, a, o) \left(R'(s, a, o) + V'^*(o) \right) \right].$$

Substitute $T'(s, a, s') = \lambda T(s, a, s')$, $R'(s, a, s') = \frac{1}{\lambda} R(s, a, s')$, $T'(s, a, o) = 1 - \lambda$, $R'(s, a, o) = 0$, and $V'^*(o) = 0$:

$$V'^*(s) = \max_{a \in A(s)} \left[\sum_{s' \in S} \lambda T(s, a, s') \left(\frac{1}{\lambda} R(s, a, s') + V'^*(s') \right) + (1 - \lambda)(0 + 0) \right].$$

The second term is 0, and distribute λ inside the sum:

$$V'^*(s) = \max_{a \in A(s)} \sum_{s' \in S} T(s, a, s') \left(R(s, a, s') + \lambda V'^*(s') \right).$$

This is exactly the original equation of the original MDP, hence

$$V'^*(s) = V^*(s) \quad \forall s \in S.$$

3.c

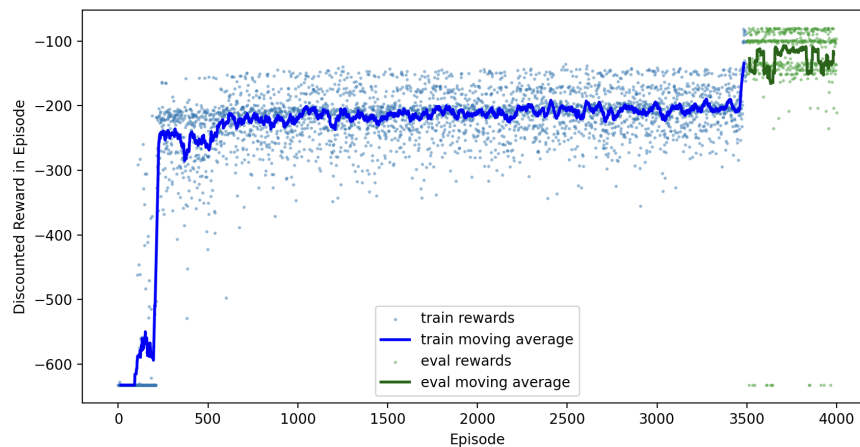


Figure 2: Value Iterations

As it can be seen in the plot above, rewards start flat and noisy while it's just exploring, then jump up once value iteration kicks in and keep trending higher as the policy improves. It can struggle when the learned model is wrong (sparse data, coarse bins), when the world is very stochastic or changes over time, or when the state space is so big/continuous that the tabular grid is either too coarse (mashes different states together) or too fine (huge table, value iteration becomes slow).

4.d

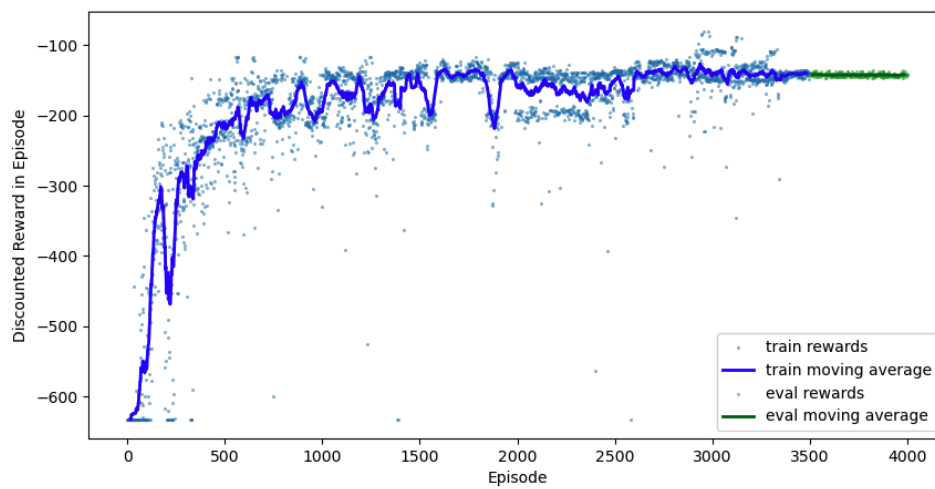


Figure 3: Tabular

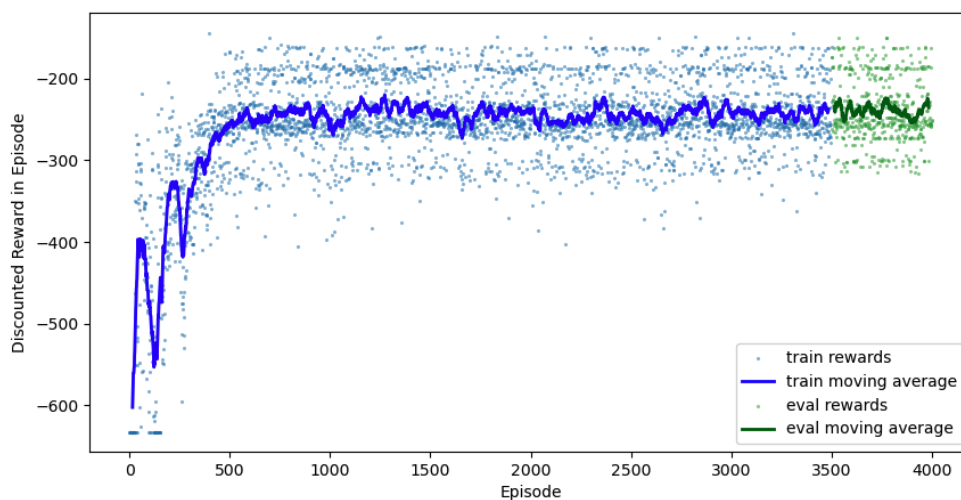


Figure 4: Function approximation

In training, tabular Q-learning climbs from around -600 towards -120 and stabilizes, showing consistent improvement and low variance by the end. The function-approximation run improves initially but flattens around -230 with higher dispersion, never reaching the tabular scores. That gap reflects that the tabular learner can fit the discretized space exactly, while the function approximation (Fourier in this case called by "train.py") blurs updates and learns a weaker policy.

4.e

Function approximation lets each update help nearby states, so you learn more from limited exploration. It stays compact and feasible in huge or continuous state spaces where a table would be massive. It trades exactness for generalization and low memory, which is often the right swap when data or space is tight. In the case for "mountaincar.py", it has a continuous, wide state space and sparse reward, function approximation lets each update generalize to nearby states, so it learns more from limited experience.

5.a

The acceleration depends on the position and the line `velocity = np.clip(velocity, -self.max_speed, self.max_speed)` is clipping the maximum speed (relative to the position) to apply, not exceeding the `max_speed`.

5.b

By increasing the maximum speed way up doesn't affect due to policy seeing the same dynamics. The force and hill physics keep velocity in the old range, so the cap is never reached.

5.c

With max speed=10000 it's basically unconstrained any action is allowed. With max speed = 0.065, ConstrainedQLearning filters out actions that would push velocity past the cap, swapping some aggressive moves for safer ones.

5.d

For the definition of the States S it would consist of information such as the distance to nearby pedestrians, the presence of traffic signals or crosswalks, and the current speed of the vehicle.

The reward function $R(s, a, s')$ encourages fast progress while (incorrectly) weakly penalizing unsafe outcomes, for example assigning a large positive reward for reaching the destination quickly without taking into account if on the way there was a signal skipped or a pedestrian accident, assigning only small negative rewards for that kind of actions

The action set $A(s)$ includes accelerating, keeping the same speed, braking, and turning.

A potentially harmful exploration policy is an ϵ greedy policy, since it occasionally selects random actions choosing actions like accelerating or ignoring traffic rules even when pedestrians are present.

This setup violates the principle of Beneficence from the Menlo Report, since it exposes pedestrians to significant risk of harm.